Week2

analysis的词语定义

“A method of studying the nature of something or of determining its essential features and their relations”.

一种研究事物本质或确定其基本特征及其关系的方法。

“A method of exhibiting complex concepts or propositions as compounds or functions of more basic ones”.

将复杂概念或命题表现为更基本概念或命题的复合或函数的一种方法。

“The evaluation of an activity to identify its desired objectives and determine procedures for efficiently attaining them”.

“对一项活动的评价，以确定其预期目标并确定有效实现这些目标的程序”。

分析也是去分析需求，分析出customer没告诉你的需求，那么

Why didn’t the customer tell you?

可能的原因：

- They may assume you know about a feature because it’s obvious to them. 他们可能会认为你知道某个功能，因为这对他们来说很明显。
- They may not think about some of the special conditions where the feature would be needed. 他们可能没有考虑到需要该特性的一些特殊情况。
- They may not know it’s necessary! 他们可能不知道这是必要的!

analysis的第一个定义是去找nature和essential，那么怎么找呢

Getting to the essentials

【笼统的三步】

Where do classes come from?【第二步中究竟怎么analyse】

这里的class我找不到一个很合适的翻译，我认为是接下来写的java代码里的类，接下来的方法是去分析所有的requirement，然后把要实现的功能总结出来，进而在代码中写成一个个的类，也就是class

Textual analysis 文本分析

- Nouns in requirements and documents. 需求和文档中的名词。

Entities and concepts 实体与概念

- From the application domain. 来自应用领域。

Experience 经验

- Previous systems 原有系统的经验

很明显，我们可以看到，上面的方法都是对requirement进行处理完善，是class的雏形，当然也可以说是“come from”

Why “Analyse”

Focus shifts to developer and system internals.

重点转移到开发人员和系统内部。

- Refining requirements. 精炼要求。

【refine这个词我觉得是对analyse的很好的概括，不止局限于利用上述三种方法对customer遗漏的点进行补充，对customer提出的冗余也要进行删改，以方便后续的coding】

Aim: precise understanding of requirements.

目标:准确理解需求。

- Process of structuring requirements: 构建需求的过程:

    - Understand 理解
    - Change 改变
    - Reuse 再利用
    - Maintain 保持

What concerns Analysis

To be used mainly by developers 主要供开发人员使用

- Using the “language” of the developer. 使用开发者的“语言”。

Provide internal view of the system. 提供系统的内部视图。

Conceptual modelling 概念模型化

- Structured by stereotypical classes and packages. 由典型的类和包构成。

Function realisation 功能实现

- Outlines how to realise the functionality within the system. 概述了如何在系统内实现功能。

Conceptual modelling 概念模型化

A conceptual model aims to identify the individual concepts (classes) which exist within a problem domain.

概念模型旨在识别存在于问题域中的单个概念(类)

It should show: (Object Oriented Analysis)

它应该显示:(面向对象分析)

- Concepts (fundamental classes) 概念(基本类)
- Attributes of concepts 概念的属性

- Operations of concepts (leave details to the design stage) 概念的操作(将细节留给设计阶段)
- Associations between concepts 概念之间的关联

Conceptual models are described using UML Class diagrams.

概念模型是用UML类图描述的。

Class and Object

Objects are entities that model some concrete or conceptual entity inside the system.

对象是对系统内的一些具体或概念性实体建模的实体。

- A class is an abstraction of an object. 类是对象的抽象。
- Every object belongs to a class, and the class of an object determines its interface (outside world view of the object). 每个对象都属于一个类，对象的类决定了它的接口(对象的外部视图)。
- The process of creating a new object belonging to a particular class is called instantiating or creating an instance of the class. 创建属于特定类的新对象的过程称为实例化或创建类的实例。

Analysis Class

Analysis classes are conceptual: 分析类是概念性的:

- High level behaviour 高级行为
- High level attributes 高级属性
- High level relationships and special requirements 高层关系和特殊需求

Analysis classes always fit in one of 3 basic stereotypes 分析类总是符合3种基本类型之一

- Entity classes 实体类
- Boundary classes 边界类
- Control classes 控制类

Entity classes

Used to model information that is long-lived and persistent

用于对长期存在且持久的信息进行建模

- Logical data structure 逻辑数据结构

Information that the system is dependent on.

系统所依赖的信息。

Store the data and define operations on the data.

存储数据并定义数据上的操作。

- Search, update, load, save, etc. 搜索、更新、加载、保存等。

Data is stored in a database and represented by entity objects in memory.

数据存储在数据库中，并由内存中的实体对象表示。

- Row in a table <=> object 表里的行 <=> 对象

Entity class example

【对于这个@Entity，可以参考：@Entity 注解 – OpenJPA 教程 (hxstrive.com)，不想看的话简单来说就是用于类上面，用来指定该类是一个实体】

Boundary classes

Used to model the interaction.

用于对交互进行建模。

Often involve receiving (presenting) information and requests from (and to)

users and external systems.

通常涉及从用户和外部系统接收(呈现)信息和请求。

- Deal with input and output, or connections with rest of system 处理输入和输出，或与系统其余部分的连接

Normally represent abstractions of user/device interface: windows, forms, communication interfaces, printer interfaces, sensors, terminals, etc.

通常表示用户/设备接口的抽象:窗口、表单、通信接口、打印机接口、传感器、终端等。

Control classes

Used to encapsulate control and coordination of the main actions and control flows.

用于封装主要操作和控制流的控制和协调。

Represent coordination, sequencing, transactionsand control of objects.

表示对象的协调、排序、事务和控制。

Deal with performing tasks, getting/setting data and coordinating behaviour.

处理执行任务，获取/设置数据和协调行为。

三种分析类结合的例子

A request from an **input form** invokes a method call on a **controller**, which uses the **data class objects** to generate a response and invokes a **boundary class object** to display the result

来自输入表单的请求调用控制器上的方法调用，控制器使用数据类对象生成响应，并调用边界类对象显示结果

Attributes

Attributes are descriptions of a particular data item maintained by each instance of a class.

属性是类的每个实例所维护的特定数据项的描述。

Every attribute has a name, a type, and if required a default initial value.

每个属性都有一个名称、一个类型，如果需要的话，还有一个默认的初始值。

- During analysis, the attribute name and type can be abstract, for example: account name, string. 在分析过程中，属性名和类型可以是抽象的，例如:帐户名、字符串。
- During later design, they should have the syntax of the target language, for example: accountName, String. 在以后的设计中，它们应该具有目标语言的语法，例如:accountName, String。

Attributes should be documented with clear, concise definitions.

属性应该有清晰、简洁的定义。

Operations

Operations are abstract specifications of a class's behaviour.

操作是类行为的抽象规范。

They have a name, a set of input parameters, and a return type.

它们有一个名称、一组输入参数和一个返回类型。

- Details of the functionality of an operation are specified textually. 操作的功能细节是通过文本指定的。

An operation should only do one thing:

一个操作应该只做一件事:

- Methods implement operations. 方法实现操作。
- Operations should be documented to state the functionality performed

by the operation. 操作应该被记录下来，以说明操作所执行的功能。

Class Relationships

A system is made up of many classes and objects. Relationships provide the pathway for communication.

系统由许多类和对象组成。关系为沟通提供了途径。

Relationships

- Association 关联
- Inheritance 继承
    - Generalisation 一般化
    - Specialisation 专业化

Association

An association is a bidirectional semantic connection between classes:

关联是类之间的双向语义连接:

- Data may flow in either direction. 数据可以向任何一个方向流动。

An association means there is a link between objects.

关联意味着对象之间存在联系。

Association Examples

按数据库的ER/EER图那边的知识理解就行

Association Multiplicity

其实跟数据库那里讲的基本一样

Multiplicity indicators 多重性指标

- **1** Exactly one
- **0..*** Zero or more
- **1..*** One or more
- **0..1** Zero or one
- **5..8** Specific range (5, 6, 7, 8)
- **4..7, 9** Combination range (4,5,6,7 or 9)

举个例子：

Inheritance

Inheritance defines a relationship among classes where one class shares the attribute(s) and/or operation(s) of one or more classes.

继承定义了类之间的关系，其中一个类共享一个或多个类的属性和/或操作。

“is–a”, “kind–of” hierarchy

" is–a ", " kind–of "等级关系【大概可以理解为谁包含谁，类似树状图，可以看下面的例子】

A subclass will inherit all attributes, operations, relationships defined in any of its superclasses.

子类将继承在其任何超类中定义的所有属性、操作和关系。

Subclass may be augmented with additional attributes and operations.

子类可以使用附加的属性和操作进行扩充。

Subclass can override attributes and operations.

子类可以覆盖属性和操作。

The key to reuse.

重用的关键。

例子

Generalisation和Specialisation

一图说清

可以理解为树状图，越往上越一般化

Analysis steps

1. Identify Entity, Boundary and Control classes 识别实体、边界和控制类
2. Identify class relationships 识别类关系
3. A conceptual class diagram 概念性类图
4. Identify attributes for each entity class 确定每个实体类的属性
5. Add constraints 添加约束

Common design characteristics

Designs have a purpose 设计是有目的的

- They describe HOW something will work. 它们描述事物如何工作。

Designs have enough information so that someone can implement them.

设计有足够的信息，所以有人可以实现它们。

There are different styles of design 有不同的设计风格

- Like different types of house architectures. 比如不同类型的房屋建筑。

Designs can be expressed at different levels of detail 设计可以在不同的细节级别上表达

- A dog house needs less detail than a skyscraper. 狗屋需要的细节比摩天大楼要少。

Our definition of “design”

Software design is the process of planning how to solve a problem through software.

软件设计是计划如何通过软件解决问题的过程。

A software design contains enough information for a development team to implement the solution. It is the embodiment of the plan (i.e. the blueprint for the software solution).

软件设计包含开发团队实现解决方案所需的足够信息。它是计划的体现(即软件解决方案的蓝图)。

Role of Design

Design transforms the analysis model into a design model that serves as a blueprint for software construction.

设计将分析模型转换为设计模型，作为软件构建的蓝图。

At this point, consideration needs to be taken for the non-functional requirements

在这一点上，需要考虑非功能性需求

e.g.

- The programming language chosen 所选择的编程语言
- Operating systems 操作系统
- Databases 数据库
- User-interfaces 用户接口

During the design phase: break down the overall task.

在设计阶段:分解整体任务。

Create a ‘skeleton’ of the system that the implementation can easily fit into.

创建一个系统的“骨架”，使实现可以很容易地融入其中。

Design Quality Guidelines

A good software design should:

- Meet the requirements 符合要求
- Be well structured: exhibit an architecture 结构良好:展示一个架构
- Be modular 是模块化的
- Contain distinct representations of data, architecture, interfaces, and components 包含数据、体系结构、接口和组件的不同表示形式
- Be maintainable 是可维护的
- Be traceable 可被追踪的
- Be well documented: represented using a notation that effectively communicates its meaning 良好的文档化:使用有效传达其含义的符号来表示
- Be efficient (when implemented) 高效(执行时)
- Be error free 无错误

一些Fundamental Concepts

- Abstraction: data, procedure, control 抽象:数据、过程、控制
- Architecture: overall structure of the software 架构:软件的总体结构
- Patterns: a proven design solution 模式:经过验证的设计解决方案
- Modularity: compartmentalization 模块化:划分
- Information hiding: encapsulation 信息隐藏:封装
- Functional independence: coupling and cohesion 功能独立性:耦合和内聚
- Refinement: elaboration of detail for all abstractions 细化:细化所有抽象的细节
- Refactoring: a reorganization technique that simplifies the design 重构:

一种简化设计的重组技术

Abstraction

分为两个：Abstract class 和 Interface

Abstract class

Defines behavior

定义行为

Can have implementation code

可以有实现代码

Cannot be instantiated

不能被实例化

A class can inherit from a single abstract class

一个类可以从一个抽象类继承

- Unless the language supports multiple inheritance 除非该语言支持多重继承

【回想上面所说的“is a”和“kind of” hierarchy，Abstract class就属于"IS-A"*[为什么不是kind of? 因为这俩表达的意思是一样的，就是主语相反]*（比如水果——香蕉、苹果、梨，水果这个大类就是Abstract class）】

Interface

和上面的Abstract class对比着看！！

Defines behavior

- Contract

Cannot be instantiated

A class can implement multiple interfaces

- In languages that support interfaces

【这里给出的是CAN-DO关系，也很好理解，从java来说，继承了接口也就是可以利用这里面的方法】

How do I know when to use an interface and when to use an abstract class?

我如何知道何时使用接口，何时使用抽象类?

- If (almost) all classes implementing the behavior would have the same code, then you can use an abstract class to implement it. 如果(几乎)实现行为的所有类都有相同的代码，那么您可以使用抽象类来实现它。

【这里就是java的知识了】

举个例子：（多态）

Encapsulation 封装

Restricting of direct access to some of an object's components

限制直接访问对象的某些组件

- Information hiding 信息隐藏

Bundling of data with the methods that operate on that data

数据与操作该数据的方法的捆绑

- Implementations of abstract data types 抽象数据类型的实现

Modularity 模块性

Separate the functionality of a program into independent, interchangeable modules

将程序的功能分离成独立的、可互换的模块

Each module contains everything necessary to execute only one aspect of the desired functionality.

每个模块只包含执行所需功能的一个方面所需的所有内容。

A module interface expresses the elements that are provided and required by the module.

模块接口表示模块提供和需要的元素。

The elements defined in the interface are detectable by other modules.

接口中定义的元素可以被其他模块检测到。

Coupling 连接

The number of dependencies between subsystems.

子系统之间的依赖数。

Indicates strengths of interconnections

表明相互联系的强度

- Tight: relatively dependent. Modifications to one is likely to have impact on others. 紧:相对依赖。对其中一项的修改可能会对其他项产生影响。
- Loose: relatively independent. Modifications to one will have little impact on others. 松散:相对独立。对其中一个的修改对其他的影响不大。

Ideally, subsystems are as loosely coupled as reasonable to minimise the impact on errors or future change.

理想情况下，子系统应该是松散耦合的，以尽量减少对错误或未来更改的影响。

关于tight coupling和loose coupling

图里很明显就能看出来连接的紧密与否

Cohesion 内聚性

The number of dependencies within a subsystem.

子系统中依赖项的数量。

【注意，coupling说的是between，而cohesion说的是within，注意区分】

A measure of the level of functional integration within a module.

模块内功能集成级别的度量。

- High: objects are related to each other and perform similar tasks. 高：对象之间相互关联，执行相似的任务。
- Low: unrelated objects. 低：不相关对象。

Ideally, a subsystem should have high cohesion.

理想情况下，子系统应该具有高内聚性。

- All parts of the component should contribute to its logical function. 组件的所有部分都应为其逻辑功能做出贡献。
- If it is necessary to change the system, then everything to do with the component is encapsulated in one place 如果有必要更改系统，那么与组件有关的所有内容都被封装在一个地方

关于high cohesion 和 low cohesion

每个变量尽量都能用在不同的方法里

Refactoring 重构

First: get the code to work.

首先:让代码正常工作。

Second: ensure that the code stays clean.

第二:确保代码保持干净。

- No duplicate code in the system 系统中没有重复代码
- The code is clean and expressive, clearly stating the intent of the code 代码干净而富有表现力，清楚地陈述了代码的意图

Refactoring 重构

- Frequently review/change code, without changing its external behaviour 经常检查/更改代码，但不改变其外部行为
- Refactoring is intended to improve nonfunctional attributes of the software 重构的目的是改进软件的非功能性属性

【就是说要让代码更简洁、更易读】

举个例子：

代码执行的函数和时间一样，但是这么写就省了一行，更简洁好看

Advantages of Object Oriented Design

Easier maintenance:

易维护

- Objects are independent. 对象是独立的。
- Objects may be understood as stand-alone entities. 对象可以理解为独立的实体。

Objects are potentially reusable components: 对象是潜在的可重用组件:

- Reuse previous developed objects 重用以前开发的对象
- Standard object 标准对象
- Inheritance 继承

For some systems, there may be an obvious mapping from real world entities to system objects.

对于某些系统，可能存在从现实世界实体到系统对象的明显映射。

Design steps

1. Based on the conceptual class diagram produced from the Analysis stage. 基于从分析阶段产生的概念性类图。
2. Identifying Class Relationships: Associations / Generalisations 识别类关系:关联/一般
3. Identify operations 确定操作
4. Describing methods 描述方法
5. Captures implementation requirements. 捕获实现需求。
6. Produce detailed design class diagram. 制作详细的设计类图。

Implementation 执行

Mapping Design to code

将设计映射到代码

Code generation should be relatively mechanical

代码生成应该相对机械

- Most of the creative work has been carried out in the analysis and design stages. 大多数创意工作都是在分析和设计阶段进行的。

Purpose of Implementation

Implement the system in terms of components (source code, scripts, binaries, executables, etc.).

根据组件(源代码、脚本、二进制文件、可执行文件等)实现系统。

The system is implemented as a succession of **small, manageable steps**.

该系统是由一系列可管理的小步骤实现的。

- The components are tested, and then integrated into one or more executables. 对组件进行测试，然后集成到一个或多个可执行文件中。

The system is distributed by mapping executable components onto nodes in the deployment model:

系统通过将可执行组件映射到部署模型中的节点来分布:

- primarily based on active classes found during design. 主要基于设计期间发现的活动类。

Component

Physical packaging of model elements, such as design classes.

模型元素的物理包装，比如设计类。

Some standard stereotypes of components include the following:

组件的一些标准构造型包括以下内容:

- <<executable>> is a program that may be run on a node 可以在节点上运行的程序
- <<file>> is a file containing source code or data 包含源代码或数据的文件
- <<library>> is a static or dynamic library 静态或动态库
- <<table>> is a database table 数据库表格
- <<document>> is a document 文档

A component **traces** the design element it implements

组件跟踪它实现的设计元素

Implementation Subsystem 实施子系统

Implementation subsystems may consist of components, interfaces, and other subsystems.

实现子系统可能由组件、接口和其他子系统组成。

It provides interfaces (exportation of operations).

它提供接口(导出操作)。

It depends on the “packaging mechanism” of the implementation environment, such as:

它取决于实现环境的“打包机制”，例如:

- A **package** in Java. 包
- A **project** in Visual Basic. 项目
- A **directory** of files in a C++ project. 目录
- A **subsystem** in an integrated development environment such as Rational Apex 子系统

图例：

注意，是“trace”的，design预想的输出应该和implementation的实际输出是一样的

Build

The software must be built incrementally in manageable steps so that each step yields small integration or test problems.

软件必须在可管理的步骤中逐步构建，以便每个步骤都会产生小的集成或测试问题。

The result of each step is called a “build”, which is an executable version of the system, normally a part of the system.

每个步骤的结果称为“构建”，它是系统的可执行版本，通常是系统的一部分。

Each build is subject to integration tests before the subsequent build is created; each build has a version control so that it is possible to go back to the previous build.

在创建后续构建之前，每个构建都要进行集成测试;每个构建都有一个版本控制，这

样就可以返回到以前的构建。

Integration Build Plan 集成构建计划

An integration build plan describes the sequence of builds required in an iteration.

集成构建计划描述迭代中所需的构建序列。

For each build, the plan describes:

对于每个构建，计划描述如下:

- The functionality that is expected to be implemented. 预期要实现的功能。
- Which parts of the implementation model are affected by the build, listing the subsystems and components required to implement the functionality expected by the build. 实现模型的哪些部分会受到构建的影响，列出实现构建所期望的功能所需的子系统和组件。

Mapping designs to code

OOP: Object–Oriented programming 面向对象编程

- Code creation in an OO language

Implementation in an object–oriented programming language (for example, Java) requires writing source code for:

在面向对象的编程语言(例如Java)中实现需要为以下内容编写源代码:

- Class definitions 类别定义
- Methods’ definitions 定义和试验方法

Class definitions

Class diagrams provide the class name, attributes, operations.

类图提供了类名、属性和操作。

- sufficient to create a basic class definition in an OO language. 足以在面向对象的语言中创建基本类定义。

It should be straightforward

应该是直截了当的

class diagram的内容与Java的对应关系

Method definitions

Interactions between objects:

对象之间的交互:

- Show the sequence of messages that are sent in response to a method invocation. 显示为响应方法调用而发送的消息序列。
- The sequences of these messages translate to a series of statements in the method definition. 这些消息的序列转换为方法定义中的一系列语句
- Parameters, return type, method decomposition. 参数、返回类型、方法分解。

Association

An association is a bidirectional semantic connection between classes.

关联是类之间的双向语义连接。

An association means there is a link between objects.

关联意味着对象之间存在联系。

↑无论是borrows还是teaches，都是两个对象之间的link/association

OO programming languages do NOT provide the concept of association.

面向对象的编程语言不提供关联的概念。

Instead, they provide:

- References: one object stores a handle to another object. 引用:一个对象存储另一个对象的句柄。
- Collections: reference to several objects can be stored and ordered. 集合:可以存储和排序多个对象的引用。
- In Java, associations have to be implemented by an adequate combination of classes, attributes and methods. 在Java中，关联必须通过类、属性和方法的适当组合来实现。

References are unidirectional:

引用是单向的:

- Direction 方向
- Multiplicity 多样性

Unidirectional one-to-one 单向一对一关联

The simplest association is a unidirectional one-to-one association.

最简单的关联是单向的一对一关联。

One direction: class A to class B

- A calls the operations of B, but B never invokes operations of A. A调用B的操作，但B从不调用A的操作。

Map this association to code using a reference from class A to class B:

使用类A到类B的引用将这种关联映射到代码:

- Add an attribute to class A with the type B. 向类A添加类型为B的属性。
    -

举个例子:

Unidirectional one-to-many 单向一对多关联

A associates with many B, B associates with one A.

A与许多B联系，B与一个A联系。

Unidirectional one to many associations can not be realised using a single reference.

单向的从一到多的关联不能使用单个引用实现。

Collections should be used:

应该使用集合:

- Ordered or not ordered 有序与否
- Array, ArrayList, List, Vector, LinkedList, Tree, Set, …
    -
    -

举个例子:

Advanced Mapping

- Inheritance → extends
- Interface → implements
- Design patterns 设计模式

Implementation Steps

Classes need to be implemented from least coupled to most coupled

类的实现需要从最少耦合到最多耦合

What is testing?

To break the system: 打破系统:

- find errors prior to delivery to the end user. 在交付给最终用户之前查找错误。

Systems are often deployed without being completely tested:

系统经常在没有完全测试的情况下部署:

- Testing is not decidable. 测试是不可确定的。
- Time and budget constraints. 时间和预算限制。

Testing perhaps is the longest process in software development cycle

测试可能是软件开发周期中最长的过程

- 40% of the timeline (typical). 时间轴的40%(典型)。

What does testing show?（Aim of testing）

The aim:

- Verify the results from the implementation stage by testing each software build. 通过测试每个软件构建来验证实现阶段的结果。
    - Internal builds 内部构建
    - Intermediate builds 中间的构建
    - Final customer or external party system software builds 最终客户或外部方系统软件构建

Who tests the software?

At the component level

在组件级别

- By the developers who developed the component. 由开发组件的开发人

员完成。

- Driven by delivery. 由交付驱动。

At the integration level 在集成级别

- By independent testing engineer/team who 由那些...的独立测试工程师/团队
    - were not involved with the construction of the system;but have a detailed understanding of the whole system. 没有参与过系统的建设，但对整个系统有详细的了解。
- Attempt to break it. 试着打破它。
- Driven by quality. 以质量为动力。

Testing goals

Goal 1: Validation testing 确认测试

- To demonstrate that the software meets its requirements: to both developer and customer 向开发人员和客户演示软件满足其需求
    - Custom software: test for every requirement
    - General software: test for all system features
- A successful validation test shows that the system operates as intended 成功的验证测试表明系统按预期运行

Goal 2: Defect testing

- To discover defects 发现缺陷
    - Failure: any deviation of the observed behaviour from the specified behaviour 故障:观察到的行为与指定行为的任何偏差
    - Error: system is in a state such that further processing will lead to a failure 错误:系统处于进一步处理将导致失败的状态

    - Defect: mechanical cause of an error 缺陷:错误的机械原因
- A successful defect test makes the system perform incorrectly and so exposes a defect in the system. 一个成功的缺陷测试会使系统不正确地执行，从而暴露系统中的缺陷。

Overall, **the goal of testing** is to build confidence: the software is good enough for operational use.

总的来说，测试的目标是建立信心:软件足够好，可以用于操作。

Testing policies

It is impossible to test every possible program execution sequence.

测试每一个可能的程序执行序列是不可能的。

Test should be based on a subset of possible test cases.

测试应该基于可能的测试用例的子集。

Testing policies define the approach to be used in selecting system tests. For example:

测试策略定义了选择系统测试时使用的方法。例如:

- All functions accessed through menus should be tested. 应该测试通过菜单访问的所有功能。
- Combinations of functions accessed through the same menu should be tested. 应该测试通过同一菜单访问的功能组合。
- Where user input is required, all functions must be tested with correct and incorrect input. 在需要用户输入的地方，所有功能都必须用正确和不正确的输入进行测试。

如何界定Good Test

A good test

- Has a high probability of finding an error 有很高的概率发现错误
    - Tester must understand the software and know where the software might fail. 测试人员必须了解软件，并知道软件可能在哪里失败。
- Is not redundant 不是多余的
    - Every test should have a different purpose. 每个测试都应该有不同的目的。
- Should be “best of breed” 应该是“最好的品种”
    - Due to time and resource limitation, only a subset of tests can be conducted. In such cases, the test that has the highest likelihood of discovering errors should be used. 由于时间和资源的限制，只能进行一部分测试。在这种情况下，应该使用发现错误的可能性最高的测试。
- Should be neither too simple nor too complex 既不简单也不复杂
    - Tests should be executed separately 测试应该单独执行

Testing process

- Unit testing (aka Component testing) → System components. 单元测试(又名组件测试)→系统组件。
- System testing → The whole system. 系统测试→整个系统。
- Acceptance testing (aka Alpha testing) → Customer’s data. 验收测试(又名Alpha测试)→客户数据。

【从图来说这三个testing process是紧密融合的】

The software testing model

【很易懂的图】

- Test case: specifications of the inputs and the expected outputs, plus statements. 测试用例:输入和预期输出的规范，加上语句。
- Test data: inputs. 测试数据:输入。
- Outputs can only be predicted by people who understand the system. 只有理解系统的人才能预测输出。

Testing Strategy

- What tests to run
- How to run them
- When to run them
- How to determine whether the testing effort is successful

举个例子

【例子中没给出第三点，也就是when，其余都给出了，但说实话，when这个确实不好界定】

Test Case Design

Design the test cases

- The goal of test case design is to create a set of tests that are effective in validation and defect testing. 测试用例设计的目标是创建一组在验证和缺陷测试中有效的测试。
- The test cases should have a high likelihood of finding errors. 测试用例应该有很高的可能性发现错误。

Tests must be conducted systematically.

测试必须系统地进行。

Test cases must be designed using disciplined techniques.

测试用例必须使用严格的技术来设计

Test Case Design Example

例子中采用了两个测试，即正确的输入和不正确的输入，这样可以测试出软件对PIN码的识别是否是正确的

Test Procedures

Identifying and structuring Test Procedures:

识别和构建测试程序:

- Reuse is important when identifying test procedures. 在确定测试过程时，重用是很重要的。
- It is vital to reuse existing test procedures but with minimal changes. 重用现有的测试过程，但是要进行最小的更改，这是至关重要的。
- Often designers will create generic test procedures that can be adapted for more specific test scenarios. 通常，设计人员将创建通用的测试过程，以适应更具体的测试场景。

Test Procedures Example

跟Test Case Design的例子是一样的，只不过Case只指那些数据，而Procedure是把整个过程/步骤写了出来

对于每一次Test的过程，要写出Test Matrix

基于上例的Test Matrix（一种总结整理所有Test的方式）

Finding defect(类似于报告的对于发现的detect的总结)

Testing: Techniques

- Test cases must be designed using disciplined techniques: 测试用例必

须使用严格的技术来设计:

- Black-box testing 黑箱测试
  - Partition testing 分类测试
  - Scenario-based testing 基于场景的测试
  - Regression testing 回归测试
- White-box testing 白箱测试
  - Basis path testing 基础路径测试

Black/White-box Testing

In general, software is tested from two different perspectives:

通常，软件测试从两个不同的角度进行:

- Black-box testing → to test software requirements 黑盒测试→测试软件需求
  - Also called **behavioural testing**. 也叫行为测试。
  - Focus is on the functional requirements of the software. 重点是软件的功能需求。
- White-box testing → to test the internal program logic 白盒测试→用于测试内部程序逻辑
  - Also called **glass-box testing** or **clear-box testing**. 也称为玻璃盒测试或透明盒测试。
  - Component level test case design. 组件级测试用例设计。
  - Internal operations are performed according to specifications, and all internal components have been adequately exercised. 内部操作按照规范执行，所有内部部件都已充分使用。

Black-box Testing

Attempt to find errors:

尝试寻找错误:

- Missing functions or incorrect functions 函数缺失或不正确
- Interface errors 接口错误
- External database access errors 外部数据库访问错误
- Behaviour errors 行为错误

图示：

异常的输入会反映在异常的输出上

Partition testing

A typical black-box testing technique

典型的黑盒测试技术

Input data and output results often fall into different classes where all members of a class are related:

输入数据和输出结果通常分为不同的类，类的所有成员都是相关的:

- Positive numbers, negative numbers, etc 正数，负数，等等

Each of these classes is an equivalence partition or domain where the program behaves in an equivalent way for each class member.

这些类中的每一个都是等价的分区或域，其中程序对每个类成员的行为都是等价的。

Test cases should be chosen from each partition.

应该从每个分区中选择测试用例。

Can be used in both component testing and system testing.

可用于组件测试和系统测试。

Example:

取数的方法：闭区间（4-10、10000-99999）取头、尾、中间值；开区间取边界

值【这里有一点瑕疵，在课后的Exercise中，老师讲的是取两个，而非上例给的一个（以第一个为例，要取2,3/10,15），取两个肯定是更妥当安全的。】

Scenario-based testing

A typical black-box testing technique

典型的黑盒测试技术

Requirements should be testable:

需求应该是可测试的:

- A test can be designed, observer can check that the requirement has been satisfied. 可以设计一个测试，观察者可以检查要求是否得到满足。

A validation testing technique where you consider each requirement and derive a set of tests for that requirement.

一种验证测试技术，考虑每个需求并为该需求导出一组测试。

Used in system testing.

用于系统测试。

Example

也算是一种分类测试，分为1, 2, >2三个scenario来进行测试

Regression Testing

For integration testing.

用于集成测试。

At each build stage, test cases will be created to test the functionality of the build.

在每个构建阶段，将创建测试用例来测试构建的功能。

As this development is incremental, as new functionality is added to the build, so there are increasing numbers of test cases.

由于这个开发是递增的，由于新的功能被添加到构建中，所以测试用例的数量也在

增加。

These test cases will be used at each stage of iteration and they make up what are known as Regression Tests.

这些测试用例将在迭代的每个阶段使用，它们组成了所谓的回归测试。

The concept is that you continue to run all of the tests for each build to ensure that earlier functionality is not broken by updates to the build

其概念是继续为每个构建运行所有测试，以确保早期的功能不会因对构建的更新而中断

Example

很好理解，就是在每次迭代时不仅完全沿用上次迭代的测试用例，还要根据新的功能添加新的测试用例

White-box testing

Goal: to ensure that all statements and conditions have been executed at least once.

目标:确保所有语句和条件至少执行了一次。

White-box testing test cases

- All independent paths within a module have been exercised at least once. 模块内的所有独立路径都至少执行过一次。
- Exercise all logical decisions on their true and false sides. 运用所有合乎逻辑的决定来判断其真伪。
- Exercise all loops at their boundaries and within their operational bounds. 在它们的边界和操作范围内执行所有循环。
- Exercise internal data structures to ensure their validity. 实践内部数据结构以确保其有效性。

Basis Path Testing

A typical white-box testing

一个典型的白盒测试

Cyclomatic Complexity 循环复杂度

- A software metric the provides a quantitative measure of the logical complexity of a program. 软件度量提供程序逻辑复杂性的定量度量。
- Defines the number of independent paths in the basis set of a program. 定义程序基集中独立路径的数目。
- Upper bound for the number of tests that must be conducted to ensure all statements have been executed at least once. 为确保所有语句至少执行一次而必须执行的测试数量的上限。

怎么判断Cyclomatic Complexity?

Cyclomatic Complexity: number of simple decisions + 1

循环复杂度:简单决策的数量+ 1

【简单来说，就是树状图可以选择接下来路径的点的数量+1】

对于上图的Cyclomatic Complexity的分析

【三个可以选择接下来路径的点(3, 5, 8)+1=4】

Testing Steps

At each step, each class should be fully unit tested using WHITE BOX testing. This involves:

在每个步骤中，每个类都应该使用白盒测试进行完整的单元测试。这包括:

- Building a test harness. 构建测试装置。
- Checking that the methods of the class behave as expected (as defined by their operation descriptions). 检查类的方法的行为是否符合预期(由它们的操作描述定义)。

- Checking that the methods are ‘robust’. 检查方法是否“健壮”。

Integration Testing using BLACK BOX Testing.

集成测试使用黑盒测试

Test Driven Development（TDD）

TDD: write tests prior to write the production code

TDD:在编写产品代码之前编写测试

TDD is a simple, short-cycled mechanism

TDD是一种简单、短周期的机制

- Write a specification, in code and in the form of a unit test. The test verifies a functional unit of your code. 用代码和单元测试的形式编写一份规范。该测试验证代码的一个功能单元。
- Demonstrate test failure. 演示测试失败。
- Write code to meet the specification. 编写代码来符合规范。
- Demonstrate test success. 演示测试成功。
- Refactor the code, to ensure that the system still has an optimally clean code base. 重构代码，以确保系统仍然有一个最佳的干净代码库。

Run all tests against the entire system at all time.

始终针对整个系统运行所有测试

TDD与Traditional Testing的区别

就是顺序不一样

TDD的流程图

JUnit

JUnit is a simple unit-testing framework for supporting TDD

JUnit是一个支持TDD的简单单元测试框架

Useful methods

Small cycle

Write a small test to assert some piece of functionality.

编写一个小测试来断言某些功能。

Demonstrate that the test fails.

演示测试失败。

Write a small bit of code to make this test pass.

编写一小段代码使测试通过。

Refactor both the test and code, eliminating duplicate concepts and ensuring that the code is expressive.

重构测试和代码，消除重复的概念，并确保代码具有表现力。